

生成对抗网络（GAN）

学习目标

GAN能做什么

什么是GAN

定义

理解

训练损失

G、D结构

案例：GAN生成手写数字图像

案例演示与结果显示

代码步骤流程

代码编写

学习目标

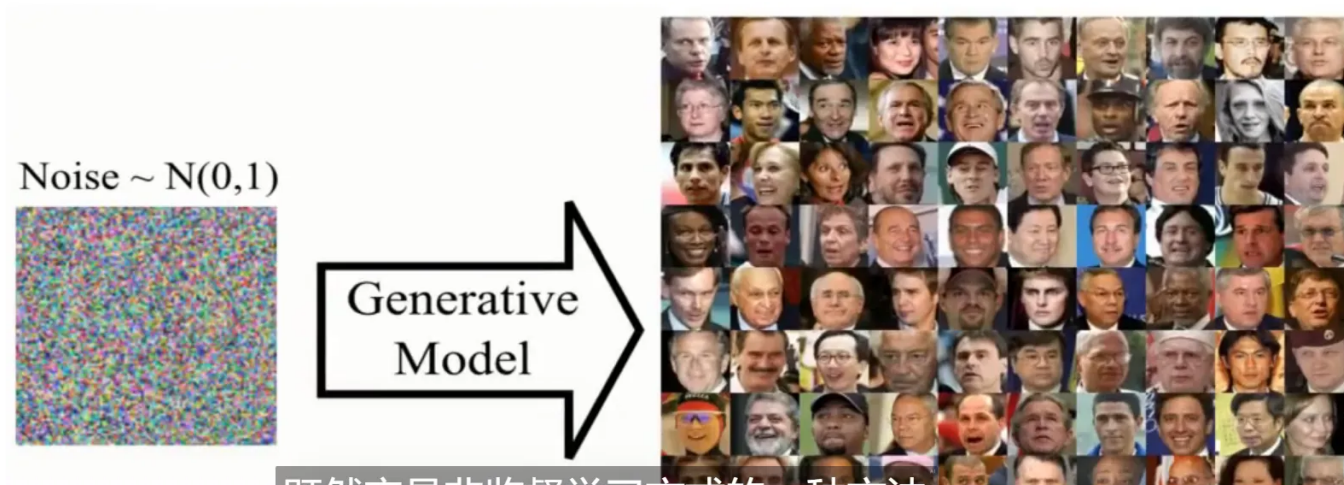
- 目标
 - 了解GAN的作用
 - 说明GAN的训练过程
 - 知道DCGAN的结构
- 应用
 - 应用DCGAN模型实现手写数字的生成

GAN能做什么

GAN是非监督式学习的一种方式，在2014年被提出

GAN主要用途：

生成以假乱真的图片

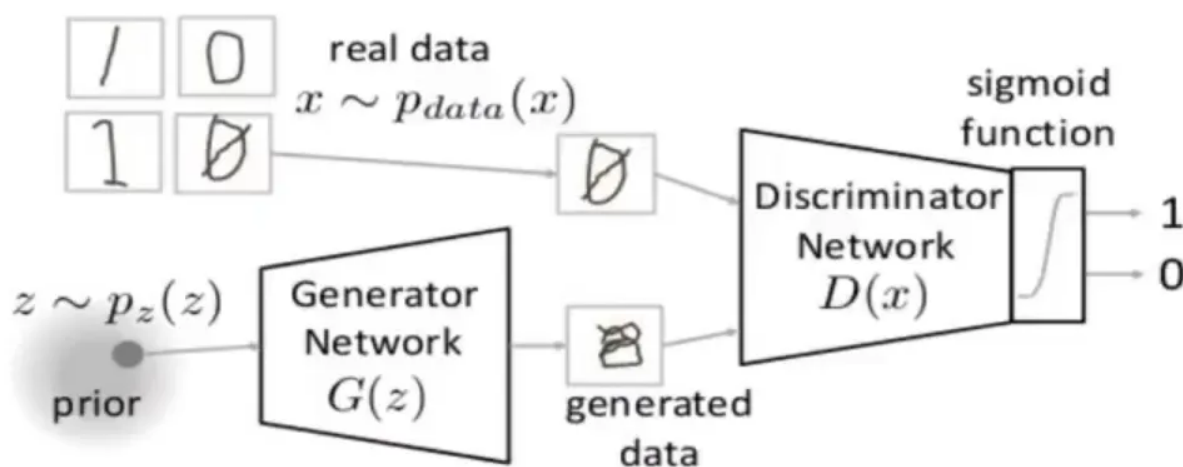


生成视频、模型

什么是GAN

定义

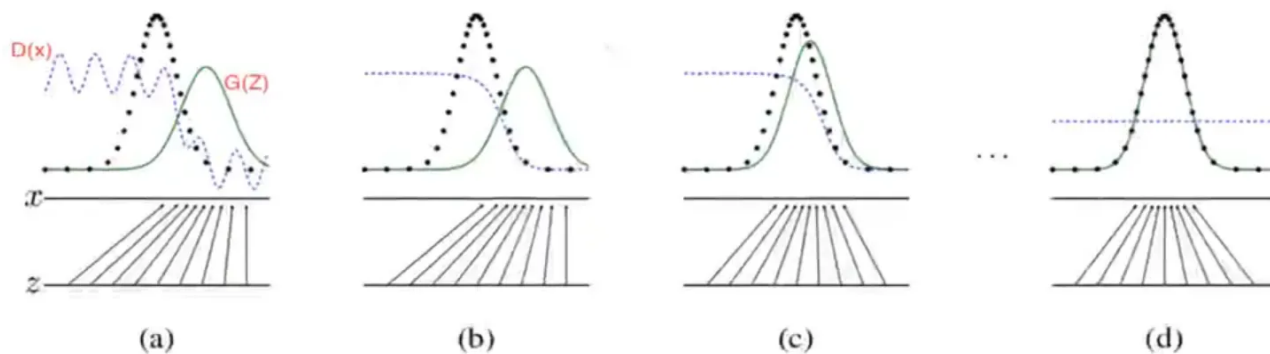
生成对抗网络(Generative Adversarial Network,简称GAN),主要结构包括一个生成器G(Generator) 和一个判别器D(Discriminator)。



- 生成器(Generator),能够输入一个向量, 输出需要生成固定大小的像素图像
- 判别器(Discriminator) ,用来判别图片是真的还是假的, 输入图片 (训练的数据或者生成的数据) 输出为判别图片的标签

理解

思想: 从训练库里获取很多训练样本, 从而学习这些训练案例生成的概率分布



- 黑色虚线：真是样本的分布
- 绿色实线：生成样本的分布
- 蓝色虚线：判别器判断的概率分布
- z 表示噪声， z 到 x 表示生成器生成的分布映射

过程分析：

1、定义GAN结构生成数据

- (a)状态处于最初始的状态，生成器生成的分布和真实分布区别较大，并且判别器判别出样本的概率不稳定

2、在真实数据上训练n epochs判别器，产生fake(假数据)并训练判别器识别为假

- 通过多次训练判别器来达到(b)样本状态，此时判别样本区分得非常显著

3、训练生成器达到欺骗判别器的效果

- 训练生成器之后达到(c)样本状态，此时生成器分布相比之前，逼近了真实样本分布。经过多次反复训练迭代之后。
- 最终希望能够达到(d)状态，生成样本分布拟合于真实样本分布，并且判别器分辨不出样本是生成的还是真实的。

训练损失

$$\min_G \max_D V(D, G) = \mathbb{E}_{\mathbf{x} \sim p_{\text{dam}}(\mathbf{x})} [\log D(\mathbf{x})] + \mathbb{E}_{\mathbf{z} \sim p_z(\mathbf{z})} [\log(1 - D(G(\mathbf{z})))]$$

- $V(G, D)$ ：项示 P_x 和 P_z 的旗景程度。
- $\max_D V(D, G)$ ：固定生成留, 尽可能地让判别最大化地判别出样本评自于真实数据还是生成的数据
- $\min_G V(D, G)$ ：固定判别器 D 的条件下得到生成器 G , 能够小化真实样本与生成样本的差异。

整个优化我们其实只看做一个部分：

- 判别器：相当于一个分类器，判断图片的真伪，二分类问题，使用交叉熵损失

对于真实样本·对数预测概率损失，提高预测的概率

$$E_{x \sim p_{data}(x)} \log(D(x))$$

对于生成样本：对数预测概率损失，降低预测概率

$$E_{z \sim p_z(z)} \log(1 - D(G(z)))$$

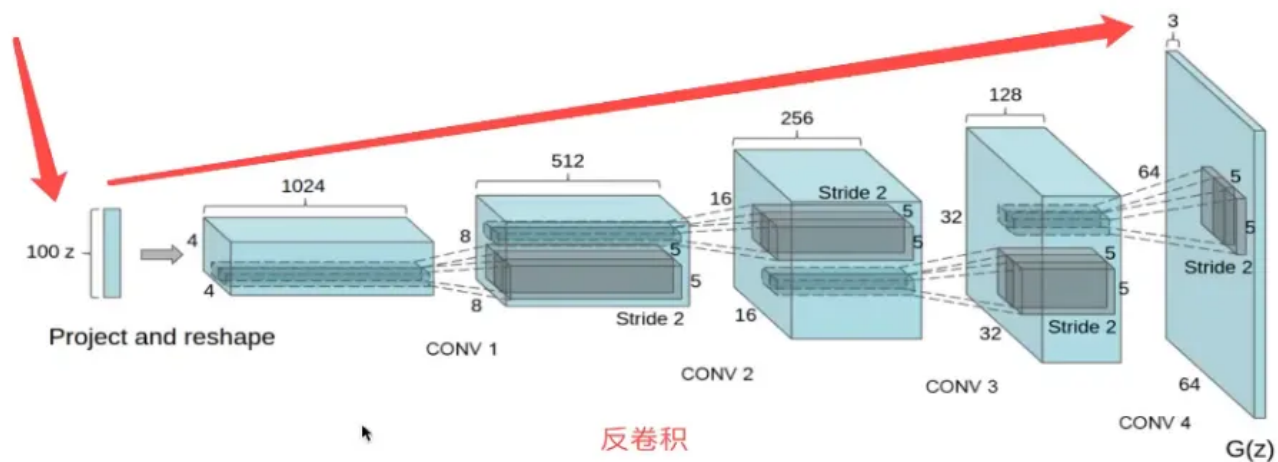
最终可以这样：

$$\begin{aligned} \text{Maximize } \tilde{V} &= \frac{1}{m} \sum_{i=1}^m \log D(x^i) + \frac{1}{m} \sum_{i=1}^m \log(1 - D(\tilde{x}^i)) \\ \text{Minimize } L &= -\frac{1}{m} \sum_{i=1}^m \log D(x^i) - \frac{1}{m} \sum_{i=1}^m \log(1 - D(\tilde{x}^i)) \end{aligned}$$

G、D结构

G、D结构是两个网络，特点是能够反向传播可导计算要介绍G、D结构，需要区分不同版本的GAN。

- 2014年最开始的模型：
 - G、D都是multilayer perceptron(MLP)
 - 缺点：实践证明训练难度大，效果不行
- 2015:使用卷积神经网络+GAN(DCGAN(Deep Convolutional GAN))
 - 改进：
 - 1、判别器D中取出pooling,全部变成卷积、生成器G中使用反卷积（下图）
 - 2、D、G中都增加了BN层
 - 3、去除了所有的全连接层
 - 4、判别器D中全部使用Leaky ReLU,生成器除了最后输出层使用tanh其它层全换成ReLU



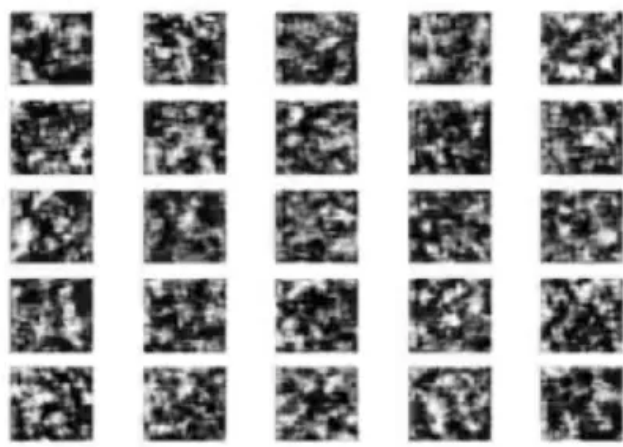
包含两个卷积网络

- 判别器：卷积网络
- 生成器：反卷积网络

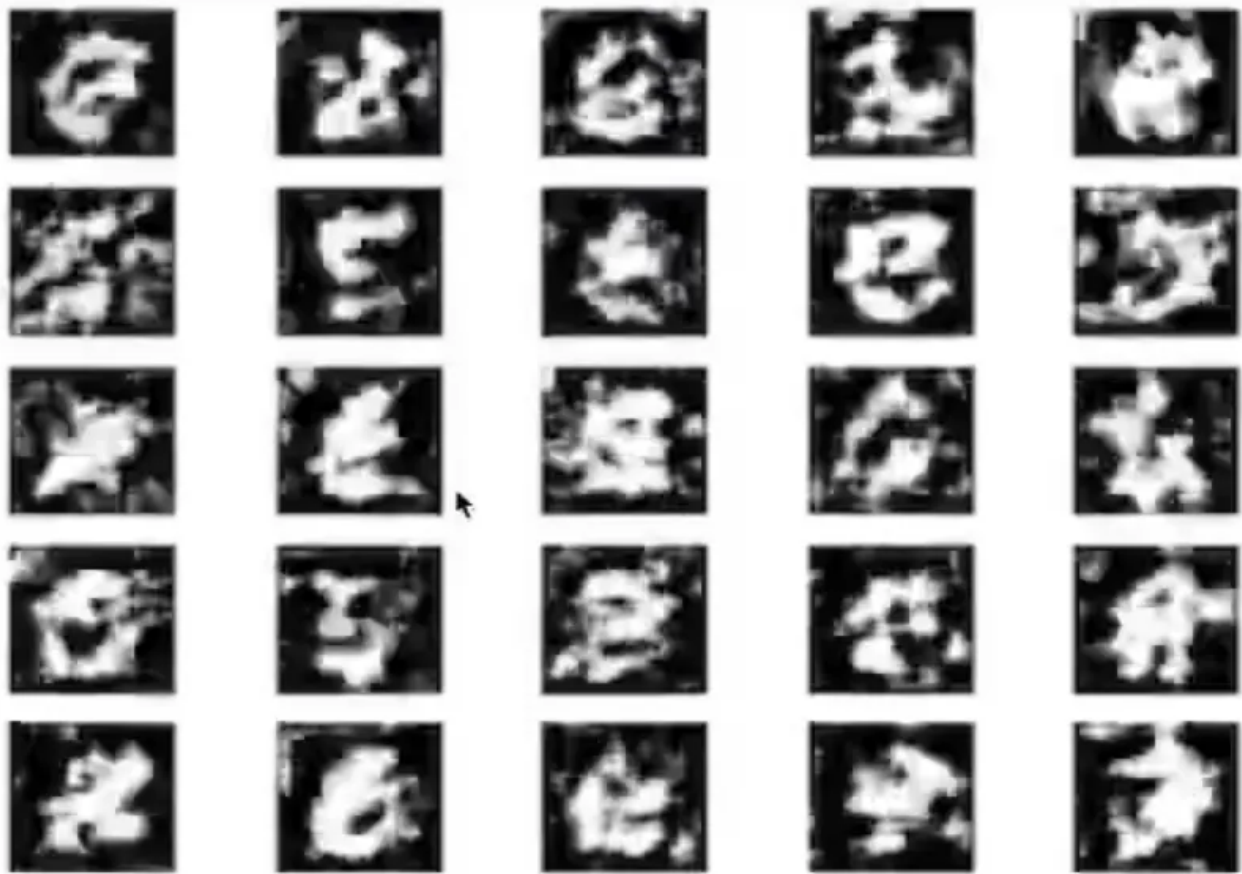
案例：GAN生成手写数字图像

案例演示与结果显示

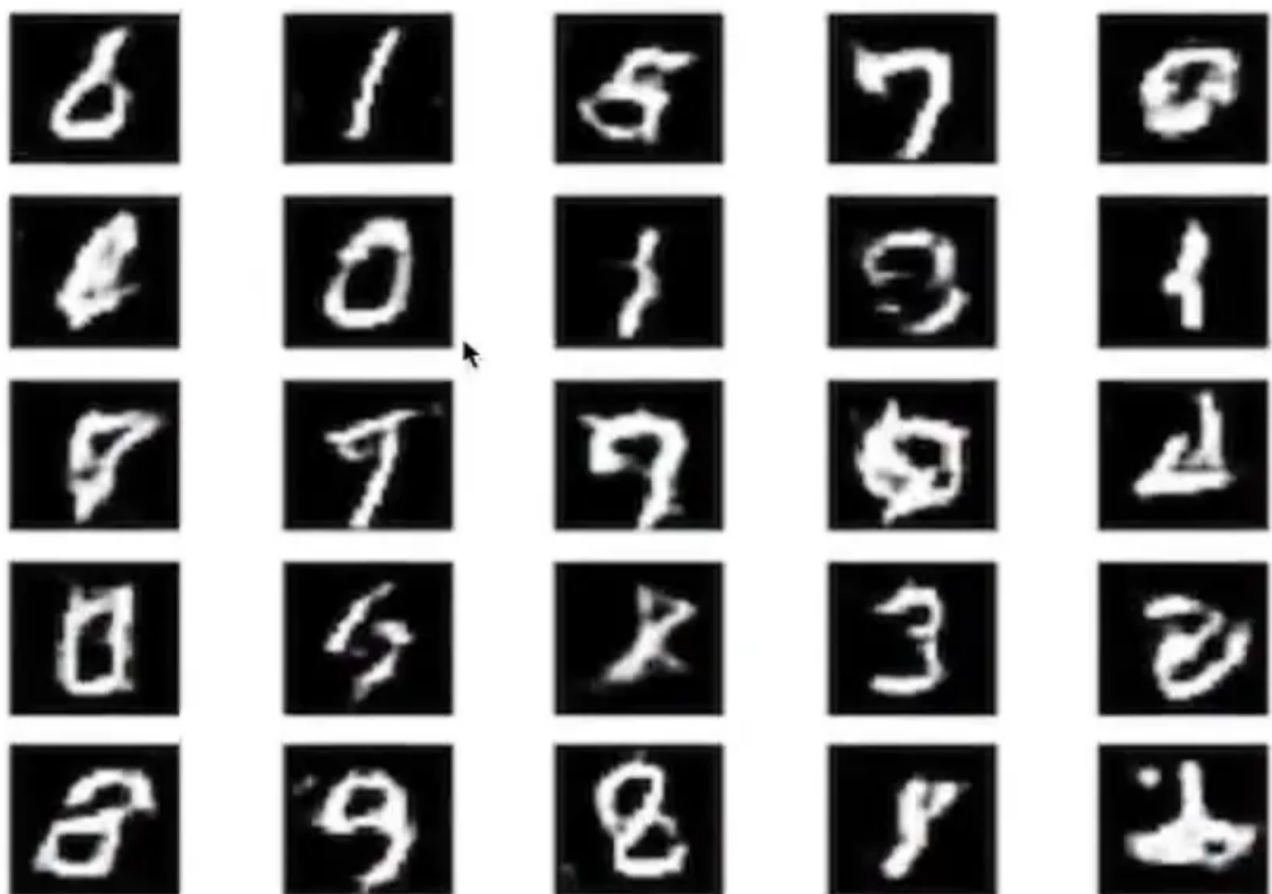
- 迭代不同的次数生成的图片效果
- 1次
- 1次



- 50



- 2000



代码步骤流程

- 初始化甘模型结构
 - o init model (个体经营)
 - o判别器: CNN,build_discriminator
 - o生成器: CNN,build_generator
- 训练过程: train(self,epochs,batch_size=128,save_interval=50)
 - 训练判别器
 - 训练生成器
 - 生成图片保存

代码编写

1, 模型类

```
class DCGAN():

    def __init__(self):
        # 输入图片的形状
        self.img_rows = 28
        self.img_cols = 28
        self.channels = 1
        self.img_shape = (self.img_rows, self.img_cols, self.channels)
```

2.初始化GAN惯型结构

- 建立d半判别器CNN结构，初始化判别器训练优化参数
- 联合建立g生成器CNN结构，初始化生成器训练优化参数
 - 输入噪点数据，输出预测的类别概率
 - 注意生成器训练时，判别器不进行训练
- 来自keras.optimizers导入Adam

```
1  def init_model(self):
2      #生成原始噪点数据大小
3      self.latent_dim=100
4      optimizer=Adam(0.0082,0.5)
5      #1、建立判别器训练参数
6      #选择损失，优化器，以及衡量准确率
7      self.discriminator=self.build_discriminator()
8      self.discriminator.compile(loss='binary_crossentropy',
9                                  optimizer=optimizer,
10                                 metrics=['accuracy'])
11     #2、联合建立生成器训练参数，指定生成器损失
12     self.generator= self.build_generator()
13
14     z = Input(shape=(self.latent_dim,))
15     img=self.generator(z)
16
17     #合并模型的损失，并且之后只训练生成器，判别器不训练
18     self.discriminator.trainable=False
19     valid=self.discriminator(img)
20     #训练生成器欺骗判别器
21     self.combined=Model(z,valid)
22     self.combined.compile(loss='binary_crossentropy',optimizer=optimiz
er)
```

定义模型的半判别图

```
from keras.layers import Input,Dense,Reshape,Flatten,Dropout from keras.layers import
BatchNormalization,Activation,ZeroPadding2D from keras.layers.advanced_activations import
```

LeakyReLU from keras.layers.convolutional import UpSampling2D,Conv2D from keras.models
import Sequential,Model

Python |

```
1  def build_generator(self):
2      model=Sequential()
3
4      model.add(Dense(128*7*7, activation = "relu", input_dim = self.latent_dim))
5      model.add(Reshape((7, 7, 128)))
6      model.add(UpSampling2D())
7      model.add(Conv2D(128, kernel_size=3, padding="same"))
8      model.add(BatchNormalization(momentum=0.8))
9      model.add(Activation("relu"))
10     model.add(UpSampling2D())
11     model.add(Conv2D(64, kernel_size=3, padding="same"))
12     model.add(BatchNormalization(momentum=0.8))
13     model.add(Activation("relu"))
14     model.add(Conv2D(self.channels, kernel_size = 3, padding = "same"))
15     model.add(Activation("tanh"))
16
17     model.summary()
18
19     noise=Input(shape=(self.latent_dim,))
20     img=model(noise)
21
22     return Model(noise, img)
```

• 3、训练模型代码

- from keras.datasets import mnist
- import matplotlib.pyplot as plt
- import numpy as np
- model.train_on_batch(feature, target)

```

1  def train(self, epochs, batch_size=32):
2      """
3      训练D、G结构
4      :param
5      epochs: 迭代次数
6      batch_size: 每次样本数
7      return:
8      """
9      # 1、加载mnist数据，处理，目标值建立
10     (X_train, _), (_, _) = mnist.load_data()
11     # 进行归一化处理
12     X_train = X_train / 127.5 - 1.
13     print(X_train.shape)
14
15     # 0,1,2,3[60000,28,28,1]
16     X_train = np.expand_dims(X_train, axis=3)
17     # 准备目标值
18     # batch_size大小真实样本的目标值1
19     # batch_size大小假样本的目标值0
20     valid = np.ones((batch_size, 1))
21     fake = np.zeros((batch_size, 1))
22     # 2、循环迭代训练
23     for epoch in range(epochs):
24         # 1、训练判别器
25         # 准备batch_size个真实样本
26         idx = np.random.randint(0, X_train.shape[0], batch_size)
27         imgs = X_train[idx]
28         # 准备batch_size个假样本, [batch_size,]
29         noise = np.random.normal(0, 1, (batch_size, self.latent_dim))
30
31         # 使用generator生成假图片
32         gen_imgs = self.generator.predict(noise)
33         # 训练
34         loss_real = self.discriminator.train_on_batch(imgs, valid)
35         loss_fake = self.discriminator.train_on_batch(gen_imgs, fake)
36
37         loss_avg = np.add(loss_real, loss_fake) / 2
38         # 2、训练生成器，停止判别器
39         # 就是去训练前面指定的combined模型
40         g_loss = self.combined.train_on_batch(noise, valid)
41         # 打印结果
42         print("迭代次数: %d, 判别器: 损失: %f, 生成器: %f" % (epoch, loss_avg[0], g_loss))

```

保存生成的图片

```
1  def save_imgs(self, epoch):
2      r, c = 5, 5
3
4      noise=np.random.normal(0, 1, (r * c, self.latent_dim))
5      gen_imgs=self.generator.predict(noise)
6      gen_imgs=0.5*gen_imgs + 0.5
7      fig, axs=plt.subplots(r, c)
8      cnt=0
9      for i in range(r):
10         for j in range(c):
11             axs[i, j].imshow(gen_imgs[cnt, :, :, 0], cmap='gray')
12             axs[i, j].axis('off')
13             cnt += 1
14         fig.savefig("./images/mnist_%d.png" % epoch)
15         plt.close()
```